

TAP — The Answer Protocol

Code Walkthrough & Resources

Project: TAP shared-world retro text adventure

September 3, 2026

Contents

1. What this document is	2
2. Repository layout	2
3. <code>common/jsonmin.c</code> — the JSON layer	2
4. <code>server/world.c</code> and <code>world.h</code> — the world model	2
5. <code>server/server.c</code> — the protocol server	3
5.1 Reactor loop	3
5.2 Per-connection state: <code>player_t</code>	3
5.3 Line framing	3
5.4 Command dispatch	3
5.5 Combat, quests, groups, logging	3
6. <code>common/log.c</code> — structured logging	4
7. <code>client-cli/cli.c</code> — the CLI client	4
8. <code>client-gui/gui.c</code> — the GTK3 client	4
9. <code>world/world.json</code> — the game world	5
10. Makefile and <code>tests/test.sh</code>	5
11. Resources	5

1. What this document is

This document is a guided walkthrough of the TAP codebase: what every file does, why it was built the way it was, how the protocol maps onto the code, and where each design decision required by the assignment (combat, quests, logging, world) is implemented. It complements `README.md`, which covers *how to build and run* the project — this document focuses on *how the code works*.

The project implements **RFC 42TAP** end-to-end: a server, a CLI client, and a GTK3 GUI client, all written in C with no dependencies beyond the C standard library, POSIX sockets, and GTK3.

2. Repository layout

```
tap/
|-- Makefile           top-level build system (all required targets)
|-- README.md          required project documentation
|-- rfc/protocol-rfc.html the RFC 42TAP specification (reference copy)
|-- world/world.json   the game world: rooms, items, NPCs, quests
|-- common/            code shared by server + both clients
|   |-- jsonmin.c / .h  minimal JSON writer + parser
|   |-- log.c / .h      structured JSON logger (server only)
|-- server/            the TAP server
|   |-- world.c / .h    world loading, validation, resource lookup
|   |-- server.c        TCP server: protocol, combat, quests, groups
|-- client-cli/cli.c    command-line client
|-- client-gui/gui.c    GTK3 graphical client
`-- tests/test.sh       automated black-box protocol test suite
```

3. common/jsonmin.c — the JSON layer

Rather than pull in an external JSON library, TAP ships a small, dependency-free JSON writer and parser (~330 lines) that is reused by the server (to build `LOOK/STATUS/QUESTS/WHO` replies and parse `world.json`) and by the GUI client (to parse those same replies back into widgets).

Writer (`sbuf_t + sb_append*`): a growable string buffer with three building blocks: - `sb_append(sb, s)` / `sb_appendf(sb, fmt, ...)` — append raw text or a `printf`-style fragment, growing the buffer as needed. - `sb_append_json_string(sb, s)` — appends `s` as a properly quoted and escaped JSON string (handles `"`, `\`, control characters).

Server code composes JSON by hand with these primitives, e.g. `build_look_json()` in `server.c` writes `{"room":{...},"players":[...], ...}` directly — no intermediate tree is built on the write side, which keeps response construction fast and allocation-free beyond the single growable buffer.

Parser (`json_parse → json_value_t* tree`): a small recursive-descent parser producing a tagged-union tree (`JV_NULL/BOOL/NUM/STR/ARR/OBJ`), with accessors `json_obj_get`, `json_arr_get`, `json_arr_len`, `json_as_str`, `json_as_num`. This is the side the **GUI client** uses: when the server replies to `LOOK` with room JSON, `gui.c` calls `json_parse()` and walks the tree to populate the room/exits/players/ items/NPCs panels.

4. server/world.c and world.h — the world model

`world.h` defines the static data model: `room_t` (id, name, description, up to 8 exits), `item_t` (id, name, description, obtainable flag, current room — empty string if held by a player or not yet spawned), `npc_t` (id, name, role enum `ROLE_DIALOGUE/ROLE_QUESTION_GIVER/ROLE_ENEMY`, home room, up to 8 dialogue lines, an embedded `quest_def_t`, and combat state `hp/max_hp/alive`).

`world_load()` reads `world.json` with the shared JSON parser and populates a `world_t` (fixed-size arrays, no dynamic allocation needed at this world size). After loading, it **validates** the world: - the `start_room` exists, - every room's every exit target exists, - every item's and NPC's room exists, - every quest's `require_item/reward_item` reference a real item id.

Any violation prints a specific error and `world_load()` returns -1, which `main()` in `server.c` treats as fatal — the server refuses to start on a broken world file rather than crash later on a bad reference.

Resource resolution (`resource_matches`, `world_find_item`, `world_find_npc`, and the room-scoped variants): implements the id/name matching rule described in README §“Protocol Implementation” — exact id, exact name, exact local-id (`npc.goblin` → `goblin`), then case-insensitive substring against name or local-id, in that priority order. This is what lets **TAKE** Herbs resolve `item.herbs` (whose name is “Healing Herbs”) and **TALK** guard resolve `npc.guard`, matching the RFC's own example commands.

5. server/server.c — the protocol server

5.1 Reactor loop

The server is a single `select()` loop (`main()`'s `while (g_running)`): one `fd_set` containing the listening socket and every connected client's socket, a 1-second timeout so the loop can check `g_running` (set by `SIGINT/SIGTERM` handlers) even when idle. On each iteration: accept new connections (`handle_accept`), then service every readable client (`handle_readable`). See README §“Architecture” for why this single-threaded design was chosen over a thread pool.

5.2 Per-connection state: `player_t`

Each of the `MAX_CLIENTS` (128) slots holds: socket fd, auth state, username, IP, current room, a fixed-size inventory array, `hp/max_hp`, combat state (`in_combat`, `combat_target`, `defending`), group id, a small array of quest states (`quest_state_t`: id, giver, completed flag), a small array of per-NPC kill counters (for defeat quests), a line-buffering input buffer (`inbuf/inlen`), and a sliding-window rate-limit counter.

5.3 Line framing

`handle_readable()` appends newly-received bytes to `player_t.inbuf`, then repeatedly extracts everything up to the next `\n` and hands it to `dispatch()`, leaving any trailing partial line in the buffer for the next `recv()`. This is the standard TCP-framing pattern required by RFC 42TAP §9.2 (fragmentation *and* coalescing must both work) and is exercised directly by `tests/test.sh` sending multiple commands back to back.

5.4 Command dispatch

`dispatch()` upper-cases the first whitespace-delimited token (the verb) and routes to one `cmd_*` handler per RFC command, plus the two extension commands `DEFEND/FLEE`. `CONNECT` is special-cased to work even before authentication; every other verb is rejected with `ERR 904 NOT_AUTHENTICATED` if the connection hasn't `CONNECTed` yet.

Every command handler follows the same shape: validate arguments/state → mutate world/player state if applicable → `sendf()` exactly one `OK/ERR` response → optionally `broadcast_room/broadcast_global/broadcast_group` zero or more `EVT` lines to other connections. This mirrors RFC §2.2's “hybrid request-response model with asynchronous event delivery”: one command always yields exactly one direct reply, while events are a separate, unordered side channel.

5.5 Combat, quests, groups, logging

These four subsystems are each documented in depth in `README.md` (§“Combat System”, §“Quest System”, and §“Server Logging”), including the exact damage formulas, quest completion rules, and the structured log

event schema. The corresponding code lives in `cmd_attack/cmd_defend/cmd_flee`, `cmd_quest/cmd_quests`, `cmd_group`, and the `log_line()` calls threaded through every handler, respectively.

6. `common/log.c` — structured logging

`log_line(level, event, fields_fmt, ...)` writes one single-line JSON object per call, with a UTC ISO-8601 timestamp, to both `stdout` and the configured log file (`logs/server.log` by default, `--log` to change it). `fields_fmt` is a `printf`-style fragment of additional JSON key/value pairs (no leading comma), letting each call site attach exactly the fields relevant to that event without a generic serialization step. Every inbound command, every outbound response, every world-state mutation, and every quest/combat milestone is logged this way — see the full field list in `README.md` §“Server Logging”.

7. `client-cli/cli.c` — the CLI client

A thin, transparent client (~150 lines): it opens a TCP connection, then runs a `select()` loop on **both** the socket and `stdin`, so it can print an EVT the moment it arrives even while the user is mid-keystroke on the next command — satisfying the “keeps receiving events while waiting for user input” requirement. Per the `README.md` “Protocol Implementation” section, this client sends raw RFC syntax directly (design option 1 from the assignment): whatever the user types is forwarded verbatim, which makes it useful for manually probing the protocol as well as for normal play.

8. `client-gui/gui.c` — the GTK3 client

The GUI (~550 lines) is built around three ideas:

Asynchronous I/O via the GLib main loop. The socket is wrapped in a `GIOChannel` and registered with `g_io_add_watch(chan, G_IO_IN | G_IO_HUP | G_IO_ERR, on_socket_data, NULL)`. This means GTK’s own event loop drives network reads — there is no polling, no second thread, and the window never freezes waiting on the network, satisfying “must remain responsive while receiving asynchronous events” without any manual threading.

A pending-verb queue for response routing. TAP responses (`OK .../ERR ...`) don’t repeat which command they answer. `send_line()` pushes the verb of every command it sends onto a small FIFO (`pending_push`); `handle_response()` pops it back off when the next `OK/ERR` line arrives, and uses it to decide how to interpret the payload — e.g. verb `"LOOK"` → parse room JSON into the room panel, verb `"STATUS"` → parse into the HP label, verb `"MOVE"/"TAKE"/"DROP"` → trigger a follow-up `LOOK + INVENTORY` refresh so the displayed room and inventory panels always reflect the latest server state (the “Updates room view automatically after TAKE/DROP” GUI requirement).

Direct widget population from parsed JSON, not a generic model/view layer: `render_look_json()` walks the parsed `LOOK` tree and directly rebuilds the exits/players/items/NPCs `GtkBox` lists (clearing old rows with `clear_box()` and creating fresh label+button rows), `render_inventory_json()` does the same for the inventory panel with per-row “Drop” buttons, and `render_status_json()/render_who_json()` update the HP and player-count labels. Because NPC role isn’t part of the `LOOK` payload, every NPC row offers all three interaction buttons (Talk/Attack/Quest) and lets the server’s error codes (`405_NPC_NOT_HOSTILE`, etc.) tell the player which action applies — this keeps the client “dumb” and fully protocol-driven, so **any** conformant server (not just this one) can drive this GUI, and vice versa, per the assignment’s “CLI and GUI clients must be interchangeable between different groups” requirement.

Chat is split into three tabs (Global/Room/Group), each its own `GtkTextBuffer`, separate from the single scrolling “Protocol Log” pane that shows every raw line sent/received — satisfying “separates the chat view from the log view”.

9. world/world.json — the game world

A 10-room map: an **8-room loop** (Square → Tavern → Market → Forest Edge → Forest Clearing → Cave Entrance → Old Bridge → Riverside → back to Square) plus **two dead-end branches** (Cave Depths off Cave Entrance, Forgotten Ruins off Old Bridge), satisfying “at least 8 interconnected rooms forming one or more loops, plus at least one optional branch” with room to spare.

Five NPCs cover the three required roles: `npc.guard` (dialogue), `npc.merchant/npc.elder` (quest-givers), `npc.goblin/npc.cave_troll` (enemies, 30 HP / 50 HP). Five items: three obtainable in the world (`item.herbs`, `item.sword`, `item.ale`), two quest-only rewards (`item.gold_coin`, `item.goblin_tooth`, spawned only on quest completion — “`room`”: null in the JSON until then, so they can never be found by exploring or duplicated). Two quests: a fetch quest (`fetch_herbs`, via the merchant) and a defeat quest (`defeat_goblin`, via the elder) — full mechanics in README.md §“Quest System”.

10. Makefile and tests/test.sh

The Makefile builds all three binaries with `gcc -std=c11 -Wall -Wextra`, and provides every required target (`deps`, `run-server`, `run-client`, `run-client-gui`, `lint`, `test`, `clean`, `fclean`, `re` — see README.md §“Building and Running” for the full table).

`tests/test.sh` is a **dependency-free black-box test suite**: it uses only bash’s `/dev/tcp` pseudo-device as a raw TCP client (no `nc`, `python`, or other tooling required), starts a dedicated server instance on a private test port, and runs 31 assertions across connection lifecycle, world exploration, item management, combat, the full quest lifecycle, and two-player chat/group interaction — see README.md §“Testing” for the full breakdown. `make test` (which depends on `make all`) is the single command needed to build and fully exercise the project.

11. Resources

- **RFC 42TAP** (rfc/protocol-rfc.html) — the normative specification this project implements; every command, event, and error code in `server.c` traces back to a specific section of this document.
- **Beej’s Guide to Network Programming** — general background on POSIX sockets, TCP framing, and the `select()` reactor pattern used by the server’s main loop.
- **GTK3 API documentation** (docs.gtk.org) — reference for `GtkTextView/GtkTextBuffer`, `GtkNotebook`, `GtkListBox`-style manual row containers, and `GIOChannel/g_io_add_watch` for async socket I/O.
- **json.org** — general reference for JSON grammar, consulted while writing the minimal hand-rolled parser/writer in `common/jsonmin.c`.

AI usage disclosure (see also README.md §“Resources”): an AI assistant (Claude) drafted every source file in this repository from the RFC and the assignment brief, and iteratively compiled and ran the code against a live server instance to catch real bugs — most notably, an initial exact-match-only resource lookup that rejected the RFC’s own example commands (`TAKE Herbs`, `TALK Baker`), fixed by adding the case-insensitive substring matching described in Section 4 above. As instructed by the assignment’s own “AI Instructions” chapter, this should be treated as a **draft to review, run, and be able to explain** — not a black box to submit unread.